Intro2CS 67101-01 Fall Semester Moed Bet Exam

משך הבחינה:
3 שעות, לא כולל תוספת זמן עבור הסטודנטים הזכאים לכך.

מבנה הבחינה:
- **חלק א'** - **10 שאלות** (4 נק' כל אחת). **יש לענות על כל השאלות.**
- **חלק ב'** - **6 שאלות תכנות** (15 נק' כל אחת). **יש לענות רק על 4 מתוך 6 השאלות**. בסוף המבחן תתבקשו לסמן אילו שאלות בחרתם.
- **סימון השאלות אשר בחרתם בבחינה** - **נקודה אחת** בונוס תינתן על סימון השאלות אשר בחרתם בחלק ב', בשאלה נפרדת, בסוף הבחינה. **שימו לב -** לא יהיה ניתן לקבל נקודה זאת בדיעבד, מרגע שהמבחן נסגר, בכל דרך שהיא וגם לא במייל. הקפידו למלא זאת כמה דקות לפני סיום הבחינה.

הנחיות נוספות:
- בשאלות התכנות עליכם לכתוב קוד **יעיל** (מבחינת זמן ריצה וזיכרון) ונקי ככל האפשר. תוכניות לא יעילות או שאינן כתובות היטב לא יקבלו את מלוא הנקודות גם אם הן עובדות נכון.
- בכל שאלות התכנות מותר לכתוב פונקציות עזר נוספות ללא הגבלה. **אך אין לשנות את חתימת הפונקציות (!)**
- אין צורך לתעד את הקוד. אין צורך ב-docstring. הערות לקוד במקומות בהם כתבתם משהו מורכב עשויות לסייע בבדיקת התשובה שלכם אך אינן הכרחיות.
- במידה ותענו על יותר מ-4 שאלות בחלק ב', ייבדקו 4 השאלות הראשונות (במקרה שלא סומנו השאלות לבדיקה).
- **הבחינה היא עם חומר סגור. אין לעשות שימוש בפלאפון מכל סוג שהוא, במחשבון, בדפי נוסחאות, או בכל חומר עזר אחר.**
- מותר לעשות שימוש במחברות טיוטא אשר יוחלקו בכיתות. אין צורך להגיש אותם בסוף הבחינה.
- בשאלות השלמת קוד בחלק א', **אין קשר בין אורך המקטע שמסמן קוד חסר, לבין כמות הקוד שצריך למלא**.
- בשאלות הקוד בחלק ב', היכן שרשום שלא ניתן לערוך את הקוד מעל השורה הנוכחית, **אין לערוך את הקוד מעל השורה הנוכחית.**
- אנא המנעו משאלות לסגל הקורס, אשר אינן קשורות ישירות להבנת השאלה. צוות הקורס לא יוכל לספק רמזים או לענות על שאלות אחרות במהלך המבחן.

בהצלחה!

Q1:

נתונה התוכנית הבאה:

```python
class A:
    def __init__(self, n: int) -> None:
        self.n = n
        self.p = None
        if n > 0:
            self.p = A(n-1)

    def replace(self) -> None:
        if self.p:
            self.p = A(self.n-1)
            self.p.replace()

a = A(n)
a.replace()
```

מהי סיבוכיות זמן הריצה של הקוד כפונקציה של n? תנו חסם הדוק ככל הניתן.

מהי סיבוכיות המקום (זכרון) כפונקציה של n? תנו חסם הדוק ככל הניתן.

- Runtime: $O(n^2)$
- Space: $O(n)$

Q2:

החליפו את סימני השאלה בקוד מטה כך שהשורה

`delay(print, *sentence)`

לא תדפיס דבר, אך השורה

`y=x()`

תדפיס את המילים הנתונות בתוך הרשימה `sentence` עם רווחים בין המילים.

```
# write the code line that replaces the ??? below
delay = ???
# so that:
sentence = ["hello", "world", "how", "are", "you"]
x = delay(print, *sentence)  # does not print anything
y = x()  # prints all the words in the sentence e.g.
# printed: "hello world how are you"
```

שימו לב: תשובתכם צריכה לעבוד עבור רשימת מילים `sentence` באורך כלשהוא, ולא רק עבור הרשימה הנתונה.

הקוד שצריך להחליף את סימני השאלה הוא:

```
lambda f, *args: lambda: f(*args)
```

Q3:

נתונה התוכנית הבאה בה הושמט הערך שמושם למשתנה lst:

```
lst = ???

def func(n):
    return 1/n[0]

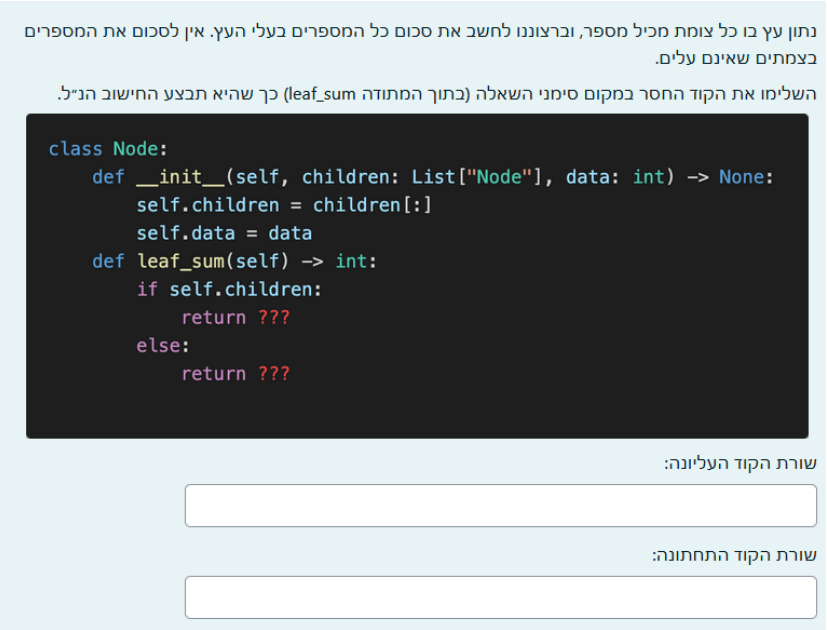
try:
    print("A", end="")
    func(lst[0])
    print("B", end="")
except ZeroDivisionError as e:
    print("C", end="")
    try:
        func(lst[1])
        print("D", end="")
    except IndexError as e:
        print("E", end="")
    finally:
        print("F", end="")
finally:
    print("G", end="")
```

מבין המחרוזות הבאות, אילו עשויות להיות מודפסות כתוצאה מריצת ההתוכנית (עבור ערך כלשהוא של lst),
ואילו הן בלתי אפשריות (כלומר לא יודפסו עבור אף ערך של lst)?

ABG
ACDG
ACEFG
ACDEFG

- "ABG" and "ACEFG" can be printed
- "ACDG" and "ACDEFG" are impossible

Q4:



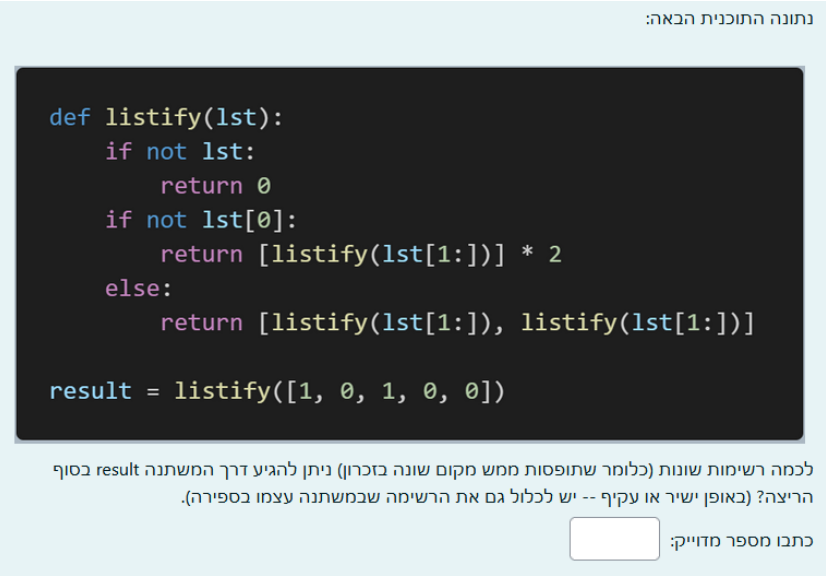
The first line:

```
return sum([child.leaf_sum() for child in self.children])
```

The second line:

```
return self.data
```

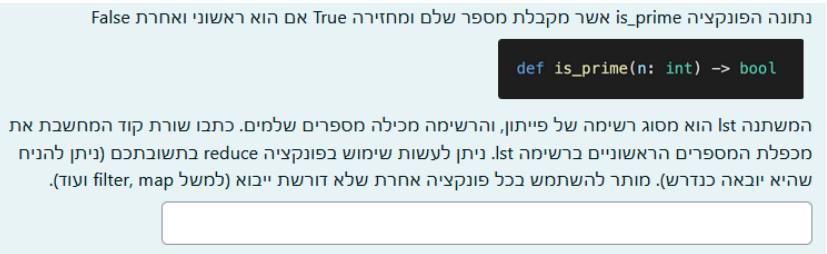
Q5:



- A "0" in the input list adds another list to what is generated recursively.
- A "1" in the input list doubles the numbers of lists created recursively, and then adds one more list. Therefore:

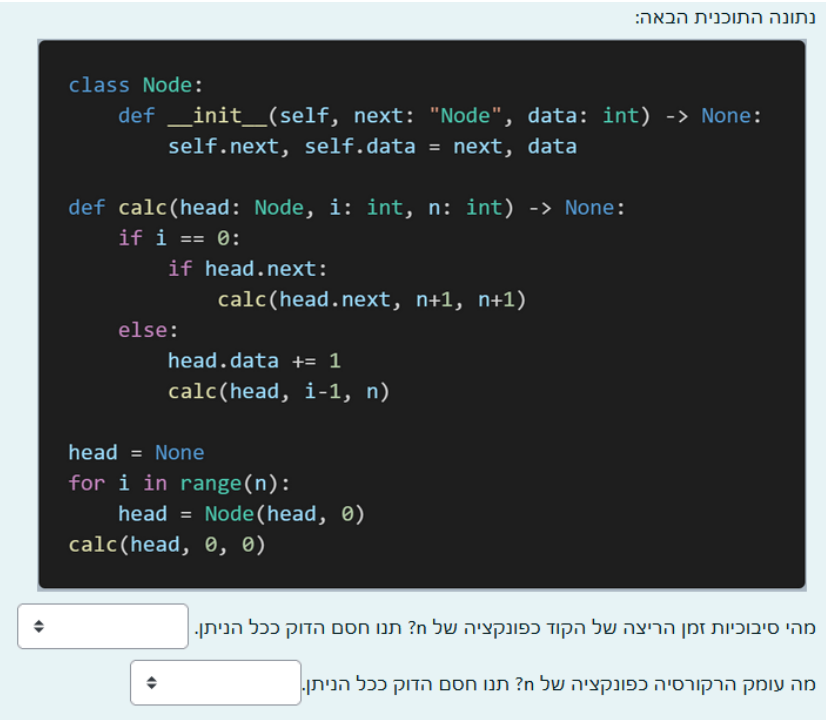
```
listify([])  # --> 0 lists
listify([0])  # --> 1 list
listify([0, 0])  # --> 2 lists
listify([1, 0, 0])  # --> 5 lists
listify([0, 1, 0, 0])  # --> 6 lists
listify([1, 0, 1, 0, 0]) # --> 13 lists
```

Q6:




```
reduce(lambda a, x: a*x if is_prime(x) else a, lst, 1)
```

Q7:



Both the runtime complexity and the recursion depth are $O(n^2)$

Q8:

נתונים שני אלגוריתמי מיון:

```python
def sort1(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i-1
        while j >= 0 and key < arr[j]:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key

def sort2(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

נניח שאנו ממיינים רשימה באורך n בעזרת כל אחד מהאלגוריתמים. על אף שלרוב אנו מעוניינים בזמן הריצה של אלגוריתם במקרה הגרוע ביותר (כלומר עבור הקלט שגורם לתוכנית לרוץ הכי הרבה זמן), בשאלה זו נתעניין דווקא **בקלט הטוב ביותר**. כלומר, בקלט שעבורו התוכנית מסתיימת הכי מהר.

מהי סיבוכיות זמן הריצה של `sort1` כפונקציה של n **במקרה הטוב ביותר**? תנו חסם הדוק ככל הניתן.

מהי סיבוכיות זמן הריצה של `sort2` כפונקציה של n **במקרה הטוב ביותר**? תנו חסם הדוק ככל הניתן.

- The first algorithm runs in time $O(n)$ on an already-sorted list (the internal while loop does not run).
- The second algorithm always runs in $O(n^2)$

Q9:

נתונה רשימה של פייתון המכילה n אובייקטים מטיפוס Book המייצגים ספרים. נרצה לחפש ספרים ברשימה בתנאים שונים. בכל אחד מהמקרים הבאים נתעניין באלגוריתם הטוב ביותר (כלומר האלגוריתם שרץ הכי מהר על הקלט הגרוע ביותר). מהי סיבוכיות זמן הריצה בכל מקרה? בכל אחד מהמקרים הנ"ל האלגוריתם המהיר ביותר עשוי לעשות שימוש גם במבני נתונים נוספים בפייתון. עליכם לתת את החסם ההדוק ביותר.

1. כאשר נחפש ספר יחיד עם כותרת נתונה במידה והספרים ברשימה מסודרים בסדר אלפביתי. סיבוכיות זמן הריצה של האלגוריתם היעיל ביותר תהיה:

2. כאשר נחפש ספר יחיד עם כותרת נתונה במידה והספרים **לא** מסודרים בסדר אלפביתי, סיבוכיות האלגוריתם הטוב ביותר תהיה:

3. כאשר נחפש n/10 כותרות ספרים מסויימות הנתונות כקלט לאלגוריתם ברשימה לא ממויינת, וכאשר רשימת הספרים המלאה בה מחפשים **לא** מסודרת בסדר אלפביתי, סיבוכיות האלגוריתם הטוב ביותר תהיה:

4. כאשר נחפש את n/10 כותרות ספרים **הראשונות (מבחינה אלפביתית)** במידה ורשימת הספרים המלאה **לא** מסודרת בסדר אלפביתי (במקרה זה לא נתונה לנו מראש רשימה של כותרות הספרים המבוקשים). סיבוכיות האלגוריתם היעיל ביותר תהיה:

1. We can do binary search in $O(n\log(n))$
2. We have to go over the entire list in the worst case, soe we need $O(n)$
3. We can place the list of all books in a dictionary, and then find every book in the shorter list in constant time. The total running time is $O(n)$
4. One option is to first sort the list and then it extract the first n/10 books. The running time of the entire algorithm is then dominated by sorting and we get $O(n\log(n))$. *However*, there is also an algorithm that finds the first n/10 books in $O(n)$ time. We accepted both $O(n)$ and $O(n\log(n))$ as correct answers for this question.

Q10

עליכם לכתוב פונקציה רקורסיבית באורך שורה אחת בלבד ולהציבה במשתנה f. הפונקציה מקבלת בתור קלט רשימה של מחרוזות ומחזירה רשימה חדשה המכילה את המחרוזות לאחר שכל האותיות בהן שונו לאותיות גדולות (upper case).

לדוגמא לקלט ['hello','world'] הפונקציה תחזיר ['HELLO', 'WORLD']

- יש להשתמש בביטוי lambda
- הפונקציה צריכה להיות באורך שורה אחת בלבד
- יש להשתמש ברקורסיה
- אין להשתמש בלולאות for, while
- אין להשתמש בפונקציות חיצונית מלבד במתודה upper של string הממירה תוים לupper case.

f=

```
f = lambda lst: [lst[0].upper()] + f(lst[1:]) if lst else []
```

Q11:

ממשו את הפונקציה `cubic_root(x, precision)` שמחשבת את x בחזקת שליש (שורש 3 של x).
תוצאת החישוב צריכה להיות קרובה לערך האמיתי עד כדי הדיוק הנתון. כלומר, אם השורש השלישי של x הוא a (כלומר מתקיים במדוייק ש a**3=x) אז הפונקציה צריכה להחזיר ערך r המקיים:

`abs(r-a) <= precision`

הערות:

- יש לתת פתרון ביעילות המיטבית. פתרונות לא יעילים יאבדו ניקוד משמעותי.
- הפונקציה מקבלת מספר ממשי x ורמת דיוק חיובית ממש (שניהם מטיפוס `float`) וצריכה להחזיר מספר ממשי מטיפוס `float`.
- יש לטפל גם ב-`x` שלילי (גם עבורו קיים שורש שלישי).
- מותר לעשות שימוש בפעולות חשבוניות רגילות `+,-,*,/` אך **אסור** לעשות שימוש בפעולת החזקה `**` או בפונקציה אחרת שממשת חזקה.
- מותר לעשות שימוש בפונקציות `abs, max, min` אך **אסור לייבא מודולים נוספים כלשהם** כולל את המודול `math`.
- ניתן להניח ש-precision שנתון הוא מספיק גדול כך שיש פתרון r המקיים את הנדרש.
- עובדה שימושית: עבור x חיובי שהשורש השלישי שלו הוא a מתקיים `a <= max(1,x)`

שימו לב: אם מחשבים בפייתון x**(1/3) כאשר x שלילי מתקבל מספר מרוכב כפתרון. זהו אכן שורש שלישי של x בשדה המרוכבים. עם זאת, עליכם להחזיר כפתרון מספר ממשי ממש (ולא מרוכב) בשונה מפעולת השורש בפייתון.

```
def cubic_root(x: float, precision: float) -> float:
    lo = 0.0
    hi = max(abs(x), 1.0)
    mid = (lo + hi) / 2

    while lo + 2*precision < hi:
        if mid*mid*mid >= abs(x):
            hi = mid
        else:
            lo = mid
        mid = (lo + hi) / 2
    return mid if x >= 0 else -mid
```

Q12:

לחברת אבטחה יש פונקציית מנעול המקבלת כקלט מפתח. המפתח הוא רשימה באורך n שכל תא בה מכיל **ספרה בודדת** כint (כלומר מספר 0 עד 9 בלבד בכל תא). הפונקציה מחזירה `True` במידה והמפתח פותח את המנעול, אחרת `False`. משימתכם לספור לכמה ערכי מפתח שונים הפונקציה תחזיר `True`. עליכם לכתוב מימוש לפונקציה count בעזרת רקורסיה.

```
def count(lock: Callable[[List[int]], bool], n: int) -> int:
```

שימו לב! בשל באג במנעול, הפונקציה מחזירה `False` אם היא מקבלת רשימה קצרה מ-n תאים שמכילה רישא של מפתח שלא ניתן להשלים לאף מפתח תקין (במקרים אחרים, עבור ערך שניתן להשלים למפתח תקין הפונקציה מחזירה True). עליכם לעשות שימוש בעובדה זו ולהימנע מבדיקת מפתחות שניתן לפסול בגלל שהרישא שלהם לא אפשרית! פתרונות שלא יעשו כך יאבדו ניקוד משמעותי.

לדוגמא, עבור פונקציית המנעול הבאה:

```
def lock(key: List[int]):
    allowed_keys = [[0, 1, 2, 3], [4, 4, 4, 4]]
    for allowed in allowed_keys:
        if allowed[:len(key)] == key:
            return True
    return False
```

הפקודה `count(lock, 4)` תחזיר 2, היות ויש בדיוק שני מפתחות מותרים באורך 4.

הערות:

- ספרה יכולה להופיע פעמיים במפתח.
- מפתחות באורך קטן או גדול מ- n לא נספרים.
- כמובן שמותר להשתמש בפונקציות עזר.
- עיקר השיטה חייב להיות backtracking מבוסס רקורסיה.
- אין להשתמש באף `import` מלבד `typing`

```python
from typing import Callable, List

def count(lock: Callable[[List[int]], bool], n: int) -> int:
    key: List[int] = []
    return helper(n, lock, key)


def helper(n: int, lock: Callable[[List[int]], bool], key: List[int]) -> int:
    if not lock(key):
        return 0
    if len(key) == n:
        return 1
    counter = 0
    for i in range(10):
        key.append(i)
        counter += helper(n, lock, key)
        key.pop()
    return counter
```

Q13:

נתונה הנוסחה הבאה (כפונקציה של פרמטר n שלם אי שלילי ושל מספר ממשי x):

$$1+\sum_{i=1}^{n}\frac{x^i}{i!}=1+\frac{x^1}{1!}+\frac{x^2}{2!}+\ldots+\frac{x^n}{n!}$$

(למי שמכיר, זהו טור טיילור המקרב את e^x עד סדר n)

עליכם לכתוב פונקציה רקורסיבית המחשבת את ערכה של הנוסחה כפי שהוגדרה למעלה.

`taylor(x: float, n: int) -> float`

- האלגוריתם צריך לרוץ בזמן ריצה **לינארי ב-n**. כלומר ב-O(n).
- ניתן להניח ש- n הנתון הוא מספר שלם אי שלילי.
- אין להשתמש בפעולות מתמטיות ואריתמטיות מלבד ל+,-,*,/.
- אין להשתמש בלולאות.
- אין לייבא מודולים כלשהם.
- עיקר השיטה חייב להיות מבוסס רקורסיה.
- כמובן שמותר להשתמש בפונקציות עזר.

```python
def taylor1(x: float, n: int) -> float:
    return helper1(x, n)[1]

def helper1(x: float, n: int) -> Tuple[float, float]:
    if n == 0:
        return 0., 1.
    else:
        vn1, vn = helper1(x, n-1)
        return vn, vn + (vn - vn1) * x / n

#### Alternatively: #####

def taylor2(x: float, n: int) -> float:
    return helper2(x, n, 1., 0)

def helper2(x: float, n: int, val: float, ind: int) -> float:
    if n == ind:
        return val
    else:
        return val + helper2(x, n, val*x/(ind+1), ind+1)
```

Q14:

כתבו את הדקורטור `repeat(n)`, עבור מספר שלם `n` כלשהוא. לאחר הקריאה הראשונה לפונקציה המקושטת, `n` הקריאות הבאות לפונקציה יריצו אותה מחדש עם אותם הפרמטרים שהיו בקריאה הראשונה (לא משנה עם אילו פרמטרים הפונקציה נקראה ב `n` הפעמים הבאות).

לאחר `n` קריאות כמו שתוארו מעלה, נתחיל "סביבת חזרות" חדשה. כלומר:
הקריאה הבאה לפונקציה המקושטת תריץ אותה עם הפרמטרים שניתנים, ואילו `n` הקריאות הבאות לפונקציה המקושטת שוב יריצו את הפונקציה עם הפרמטרים שנתנו בקריאה הראשונה בסבב.
וכך הלאה.

הערות:

- על הדקורטור לתמוך בפונקציה גנרית, כלומר לא להניח אילו פרמטרים היא מקבלת ומה היא מחזירה.
- שימו לב, שגם כשהפונקציה מופעלת עם אותם פרמטרים שוב, היא רצה ממש ומחשבת מחדש את התוצאה (ולא שומרת תוצאות קודמות בזכרון).
- הניחו שהמספר `n` הוא שלם ואי-שלילי.
- אין לייבא מודולים כלשהם בעת פתרון השאלה.

```python
@repeat(2)
def f(n):
    sum = 0
    for i in range(n):
        sum += i**2
    return sum

f(3) # returns 5 (0**2 + 1**2 + 2**2)
f(4) # returns 5 (due to repeat decorator)
f(5) # returns 5 (due to repeat decorator)
f(4) # returns 14 (0**2 + 1**2 + 2**2 + 3**2)
f(10) # returns 14 (due to repeat decorator)
f(100) # returns 14 (due to repeat decorator)
f(2) # returns 1 (0**2 + 1**2)
f(100) # returns 1 (due to repeat decorator)
f(1000) # returns 1 (due to repeat decorator)
```

```python
def repeat(n: int):
    def decorator(f):
        reps = [0]
        repeated_args = [None]
        repeated_kwargs = [None]

        def wrapper(*args, **kwargs):
            if reps[0] == 0:
                repeated_args[0] = args
                repeated_kwargs[0] = kwargs
                reps[0] = n
            else:
                reps[0] -= 1
            return f(*repeated_args[0], **repeated_kwargs[0])
        return wrapper
    return decorator
```

Q15:

נתונה מחלקה Node המתארת עץ בינארי. **"האב הקדמון המשותף הנמוך ביותר"** של שני צמתים בעץ הוא הצומת הנמצא הכי רחוק מהשורש כך ששני הצמתים הנתונים נמצאים בתת העץ שלו.

נתון עץ בינארי שבו בכל צומת מספר שלם. אף מספר בעץ לא חוזר פעמיים, וכל צומת מוגדר על ידי המחלקה Node בקוד מטה.
ממשו את הפונקציה lowest_ancestor שחתימתה היא:

lowest_ancestor(root: Node, num1: int, num2: int) -> int

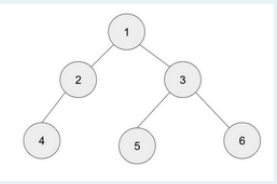
הפונקציה מקבלת כקלט את שורש העץ ושני מספרים, ומחזירה את המספר שנמצא בצומת האב הקדמון המשותף הנמוך ביותר.

הערות:

- ניתן להניח ששני המספרים הנתונים נמצאים בעץ.
- המספרים num1, num2 עשויים להיות בתת העץ אחד של השני, או אפילו יכולים להיות אותו מספר.
- ניתן להניח שכל מספר אכן מופיע בעץ פעם אחת לכל היותר.

לדוגמא, עבור העץ המתואר בציור מטה,

- עבור קריאה עם המספרים 5,6 הפונקציה תחזיר 3.
- עבור קריאה עם המספרים 4,5 הפונקציה תחזיר 1.
- ועבור קריאה עם המספרים 4,2 הפונקציה תחזיר 2.




```
class Node:
    def __init__(self, num: int, left: Optional["Node"], right: Optional["Node"]):
        self.num = num
        self.left = left
        self.right = right

# The code above this line was provided.

def lowest_common_ancestor(root: Node, node1: int, node2: int) -> int:
    path1 = find_path_to(root, node1)
    path2 = find_path_to(root, node2)
    if len(path1) > len(path2):
        path1, path2 = path2, path1
    for i in range(1, len(path1)):
        if path1[i] != path2[i]:
            return path1[i-1]
    return path1[-1]


def find_path_to(root: Node, num: int) -> List[int]:
    if root.num == num:
        return [num]
    path = []
    if root.left is not None:
        path = find_path_to(root.left, num)
    if not path and root.right is not None:
        path = find_path_to(root.right, num)
    if path:
        path = [root.num] + path
    return path
```

Q16:

כתבו מחלקה `UndoList` שממשת ממשק דומה באופיו לרשימה רגילה של פייתון. בנוסף לפעולות כגון הוספת ערכים, מחיקת ערכים, ושינוי ערכים ברשימה ניתן גם לבצע `undo` שמבטל את הפעולה האחרונה שבוצעה על הרשימה.

ממשק המחלקה:

```
lst = UndoList() #מייצר רשימה חדשה ריקה

lst.append(value) #מוסיף תא בסוף הרשימה ושם בו את הערך הנתון

lst.set(index, value) #קובע את הערך השמור בתא שבאינדקס הנתון

lst.delete(index) #מוחק את התא עם האינדקס הנתון. גודלה של הרשימה קטן ב-1

lst.get(index) #מחזיר את הערך השמור בתא עם האינדקס הנתון

str(lst)  #מחזיר ייצוג מחרוזת של הרשימה הזהה לייצוג הרשימה של פייתון

lst.undo() # מבטל את הפעולה האחרונה שבוצעה על הרשימה
```

דגשים נוספים:

- מלבד המתודה `get` והמרה למחרוזות, כל שאר המתודות מחזירות `None`.
- ניתן לקרוא ל`undo` מספר פעמים והיא תבטל יותר מפעולה אחת (ראו דוגמא למטה). הפעולות שניתן לבטל הן `append, delete, set`. הקריאה ל- `undo` עצמה אינה פעולה שניתן לבטל.
- בכל מקרה ניתן להניח שהאינדקסים שניתנים הם אי שליליים, וחוקיים (כלומר בגבולות הרשימה). אם ניתן ערך לא חוקי ההתנהגות לא מוגדרת (יכולה גם להיווצר שגיאה פנימית ואין צורך לטפל בה).
- ניתן להניח שלא יקראו ל`undo` יותר פעמים ממספר הפעולות שבוצעו עד כה ולא בוטלו.
- פורמט ההמרה למחרוזות הוא בדיוק הפורמט של פייתון: הרשימה תופיע עם סוגריים מרובעים ועם פסיקים ורווחים מתאימים בין האיברים השונים שלה. ראו דוגמא מטה.
- שימו לב: חשוב לשמור על יעילות בזכרון ככל שניתן.
- המתודה get צריכה לעבוד בסיבוכיות זמן ריצה של O(1).

<u>דוגמא לשימוש במחלקה:</u>

```
lst = UndoList() # creates an empty list
print(lst)  # prints: []
for i in range(5):
    lst.append(i)  # append values to the list
print(lst)  # prints: [0, 1, 2, 3, 4]

lst.undo()  # cancels appending the value 4 to the list
print(lst)  # prints: [0, 1, 2, 3]

lst.set(0, "A")  # sets the value at index 0 to "A"
print(lst)  # prints: ['A', 1, 2, 3]
lst.undo()  # cancels setting cell 0 to "A"
print(lst)  # prints: [0, 1, 2, 3]
lst.undo()  # cancels appending 3 to the list
print(lst)  # prints: [0, 1, 2]
lst.delete(1) # deletes the item at cell 1
print(lst)  # prints: [0, 2]
print(lst.get(1)) # prints: 2
```

```python
from typing import List, Any, Callable

class UndoList:
    def __init__(self) -> None:
        self.__lst: List[Any] = []
        self.__undo_buffer: List[Callable[[], Any]] = []

    def append(self, value: Any) -> None:
        self.__lst.append(value)
        self.__undo_buffer.append(lambda: self.__lst.pop())

    def set(self, index: int, value: Any) -> None:
        old_val = self.__lst[index]
        self.__lst[index] = value
        self.__undo_buffer.append(lambda: self.__lst.__setitem__(index, old_val))

    def delete(self, index: int) -> None:
        old_val = self.__lst.pop(index)
        self.__undo_buffer.append(lambda: self.__lst.insert(index, old_val))

    def undo(self) -> None:
        self.__undo_buffer.pop()()

    def get(self, ind: int) -> Any:
        return self.__lst[ind]

    def __str__(self) -> str:
        return str(self.__lst)
```

In [ ]: